テスト中心主義

渡邉 洋平

概要

この論文では、ソフトウェア開発におけるテスト駆動開発 (Test-Driven Development: TDD) の考え方を一般化し、概念・命題・理論をテストによって理解する枠組みを提案する。

TDD では、プログラムの仕様をテストによって記述し、そのテストに対する振る舞いによってプログラムの正しさを判断する。この考え方はソフトウェア開発に限らず、より一般的な認識の枠組みとして捉えることができる。

哲学においても、これに近い考え方が存在する。プラグマティズムでは、パースが観念の意味をその実践的帰結によって明らかにするべきであると主張した。また科学哲学では、ポパーが科学理論の条件として反証可能性を提案し、理論は観測によって誤りである可能性を持たなければならないとした。

この論文ではこれらの考え方を統一的に捉え、対象をテストによって理解する立場を**テスト中心主義 (Test-centrism)** と呼ぶ。

テスト中心主義では、テストは順序をもつ有限列として扱われ、各テストは

- 入力
- 観測値 (存在する場合) または未定義 (オープン)

から構成される。

この枠組みにおいて、テストに関する次の二つの関係を区別する。

- **識別可能性**: テストの入力に対して対象の振る舞いが定義されること
- **検証可能性**: 対象の振る舞いと観測結果を比較できること

これらは対象の種類 (概念・命題・理論) とは独立に定義される。したがって本稿では、あらゆる対象に対して識別と検証の両方を統一的に扱う。

さらに、対象の意味はテスト列に制限された振る舞いとして定義される。またこの枠組みによって、理解やリファクタリングといった概念も同一の構造のもとで形式化できることを示す。

テスト中心主義 (Test-centrism) を圏論的に形式化する。テスト列を対象とする圏を構成し、対象の振る舞いは反変関手

$$F_T : \mathcal{X}^{op} \to \mathbf{Set}$$

として表される。このとき対象の意味は presheaf として与えられ、Test-Centrism 圏

$$\mathbf{TC} = \mathbf{Set}^{\mathcal{X}^{op}}$$

は presheaf トポスとなる。

このトポスの内部論理は直観主義論理であり、さらに Kripke 意味論として解釈できる。そこではテスト列が世界に対応し、真理はテスト拡張に対して安定な性質として理解される。

本稿の枠組みにより、

- 概念 (実践的意味)
- プログラム (計算的対象)
- 科学理論 (予測体系)

がすべて同一の構造、すなわち「テストに対する応答」として統一的に記述できることを示す。

目次

1 はじめに 5

2 TDD 5
2.1 テスト駆動開発 5
2.2 リファクタリング 6
2.3 テストとは何か 6
2.4 TDD 開発の例 7
2.4.1 STEP 1 — 最初のテスト 7
2.4.2 STEP 2 — テスト追加 8
2.4.3 STEP 3 — 失敗するテスト 8
2.4.4 STEP 4 — 入力順序の問題 9
2.4.5 STEP 5 — 一度では割り切れない場合 10
2.4.6 STEP 6 — 互いに素 10

3 プラグマティズムの格率 12

4 反証可能性 12

5 テスト中心主義の定式化へ向けてのいくつかの例 13
5.1 最大公約数 13
5.1.1 振る舞い関数 13
5.1.2 テスト 14
5.1.3 テスト列 14
5.1.4 テスト結果 15
5.2 プラグマティズムの例 16
5.3 科学理論の例 17

6 テスト中心主義 18
6.1 基本アイデア 18
6.2 テストの役割 18
6.2.1 構成のためのテスト 18
6.2.2 理解のためのテスト 18

7 定式化 18
7.1 公理 1(対象) 19
7.2 公理 2(振る舞い関数) 19
7.3 公理 3(テスト) 19
7.4 公理 4(テスト列) 19
7.5 公理 5(意味) 20

7.6 公理 6(検証可能性) 20
7.7 公理 7(識別同値性) 20
7.8 公理 8(検証同値性) 20
7.9 公理 9(理解) 21
7.10 公理 10(複雑度) 21
7.11 公理 11(リファクタリング) 21

8 **例** 22
8.1 最大公約数: 検証可能な対象の例 22
8.2 プラグマティズムの格率: 硬い 24
8.3 科学理論: 反証可能性 25

9 **圏論での定式化** 26
9.1 公理 1. テスト圏 26
9.2 公理 2. 振る舞い関手 27
9.3 公理 3. 意味 27
9.4 公理 4. 識別同値 28
9.5 公理 5. 検証 28
9.6 公理 6. Test-Centrism 圏 28
9.7 Test-Centrism での Yoneda の定理 29
9.7.1 定理 29
9.8 理解の圏論的解釈 29

10 **トポスの内部論理** 29
10.1 真理値対象 30
10.2 sieve の意味 30
10.3 命題の意味 30
10.4 真理の意味 31
10.5 論理結合子 31
10.5.1 AND(論理積) 31
10.5.2 OR(論理和) 31
10.5.3 含意 32
10.5.4 否定 32
10.6 排中律が成立しない理由 32
10.7 哲学的解釈 33
10.8 科学哲学との対応 33

11 **Kripke 意味論** 33
11.1 Kripke フレーム 34
11.2 強制関係 (forcing) 34
11.3 論理結合子 34

11.3.1 AND(論理積) 34
11.3.2 OR(論理和) 35
11.3.3 含意 35
11.3.4 否定 35
11.4 トポスとの一致 35
11.5 哲学的意味 36

12 **社会理論** 36

13 **結論** 36

1　はじめに

この論文では、テスト駆動開発 (Test-Driven Development: TDD) の考え方を一般化し、概念・命題・理論を理解するための枠組みとしての**テスト中心主義 (Test-centrism)** を提案する。

TDD では、ソフトウェアの仕様をテストによって記述し、テストを通じてプログラムの振る舞いを定義する。この考え方はソフトウェア開発に限らず、より広い範囲に適用できる。

哲学においても、似た考え方が存在する。プラグマティズムでは、C.S. パースが**観念の意味はその実践的帰結にある**と述べた。また科学哲学では、サー・カール・ポパーが**科学理論は反証可能でなければならない**と主張した。

これらの立場は対象こそ異なるが、共通して次の考えを含んでいる。

> **コードにせよ観念にせよ理論にせよ、**
> **すべての対象は、それを区別するテストによって理解される。**

この論文ではこの考えを一般化し、

- 概念
- 命題
- 理論

のすべてに適用できる枠組みとして**テスト中心主義**を提示する。

2　TDD

2.1　テスト駆動開発

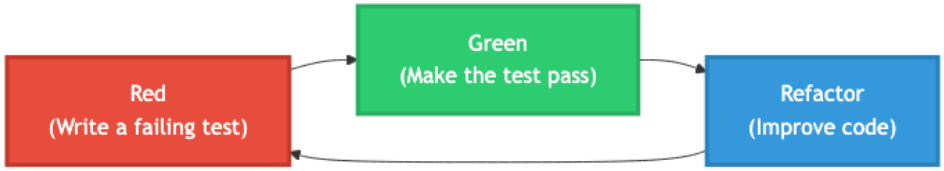
テスト駆動開発 (Test-Driven Development: TDD) はソフトウェア開発の手法の一つである。通常の開発プロセスは「仕様 →実装 → テスト」であるのに対して、TDD では順序を逆転させる。つまり、「仕様→ テスト → 実装」として、テストを先に書く。

TDD の基本原則は次の通りである。

> - 自動化されたテストが失敗したときのみ、新しいコードを書く。
> - 重複を除去する。
>
> ベック『テスト駆動開発』ix ページ

1 つ目の原則は言い換えれば、**テストが失敗するまで実装しない**ということである。

TDD の有名なマントラは「Red → Green → Refactor」である (前掲 x ページ)。



1. Red：テストを書く (失敗する)
2. Green：最小の実装で成功させる
3. Refactor：振る舞いを変えずにコードを改善する

TDD の重要な特徴は**仕様をテストで記述する**という点にある。そうすることで、まず**何を**実現したいかを明確にして、その後で**どうやって**実現するかを考えることができるようになる。

Martin Fowler は次のように述べている。

> テストを書くことで、機能を追加するためにすべきことは何かを自問することになります。また、テストを書くことで、実装よりもインタフェースに集中することになります (これは常によいことです)。コーディングが完了する時点も明確になります。それは、テストがうまくいったときです。
> 『リファクタリング』第 2 版 p.91

また、Freeman と Pryce は次のように述べている。

> 第一に、テストから先に書き始めるということは、まず**何を**実現したいかを記述し、その後で**どうやって**実現するかを考えるということだ。「何を」に集中することで、対象となるオブジェクトの抽象度を正しいレベルに保てるようになる。もしユニットテストの意図がはっきりしていないとしたら、それは複数の概念を混同しているからだろう。そうだするれば、コーディングを始める準備ができていないことになる。また、テストから先に書き始めれば、情報隠蔽もやりやすくなる。オブジェクトの外部から見えるようにすべきなのは何かを決めなければならないからだ。
> p.61『実践テスト駆動開発』

2.2 リファクタリング

Martin Fowler の『リファクタリング』ではリファクタリングを次のように定義している。

> リファクタリング（名詞）外部から見たときの振る舞いを保ちつつ、理解や修正が簡単になるように (to make it easier to understand and cheaper to modify)、ソフトウェアの内部構造を変化させること。
> 『リファクタリング』第 2 版 p.45

つまり、リファクタリングとは「振る舞いは変えない。内部構造だけを変えて、より理解しやすくする」変換である。

重要なのは**テストによって振る舞いの同一性が保証される**ことである。

テストが成功する限り、プログラムは**同値**と見なされる。

2.3 テストとは何か

『The art of software testing』ではテストを次のように定義している。

> Testing is the process of executing a program with the intent of finding errors.(テストとはエラーを見つけるつもりでプログラムを実行する過程である。)
> The art of software testing p.6

つまり、テストとは「エラーを見つける目的でプログラムを実行する過程」のことである。

2.4 TDD 開発の例

テスト駆動開発の例として最大公約数のプログラムを考える。つまり、

```
与えられた任意の 2 つの自然数 a, b に対して、
その最大公約数 gcd(greatest common divisor) を取得する。

入力: a, b
出力: gcd
```

を考える。

最大公約数を求めるアルゴリズムはいくつもある。最も有名なのがユークリッドの互除法と呼ばれるものであるが、単純な割り算探索 (brute-force) もある。

ここでは テスト駆動開発の手順に従ってプログラムを構築する。

2.4.1 STEP 1 — 最初のテスト

TDD ではまずテストを書くことから始める。最初のテスト X_1 を

- 入力:

$$i_1 = (1, 1)$$

- 期待値:

$$o_1 = 1$$

とする。すなわちテスト

$$X_1 = (i_1,\, o_1) = ((1, 1), 1)$$

を作る。

```python
# `tests/tdd_gcd_tutorial/gcd1/test_1.py`
from src.tdd_gcd_tutorial.gcd1 import gcd1

def test_t1():
    assert gcd1(1, 1) == 1
```

次に、このテストを通す最小の実装を書く。

```python
# `src/tdd_gcd_tutorial/gcd1.py`
def gcd1(a: int, b: int) -> int:
    # Return 1.
    return 1
```

テスト X_1 は**成功**する。
1 を戻す自明な実装なので、リファクタリングは不要である。

2.4.2 STEP 2 — テスト追加

次のテスト X_2 を追加する。

- 入力:

$$i_2 = (3, 1)$$

- 期待値:

$$o_2 = 1$$

すなわちテスト

$$X_2 = (i_2,\, o_2) = ((3, 1), 1)$$

である。このとき、$gcd1(i_2) = o_2 = 1$ であるので、`gcd1` は X_1, X_2 の両方で**成功**する。したがって、**実装は変更しない。**

2.4.3 STEP 3 — 失敗するテスト

次に

$$X_3 = (i_3, o_3) = ((24, 8), 8)$$

を追加する。
このとき、

$$gcd1(i_3) = 1 \neq 8 = o_3$$

である。よって、テスト X_3 は**失敗**する。

```
FAILED tests/tdd_gcd_tutorial/gcd1/test_3.py::test_t3 - assert 1 == 8
```

TDD では「失敗したテストを通す最小の実装を追加する」ので、次の実装を作る。

```python
# src/tdd_gcd_tutorial/gcd2.py
def gcd2(a: int, b: int) -> int:
    # Return the smaller number of a and b.
    return b
```

gcd2 はテスト

$$X_1, X_2, X_3$$

すべて**成功**する。

2.4.4 STEP 4 — 入力順序の問題

次のテストを追加する。

$$X_4 = (i_4, o_4) = ((8, 24), 8)$$

を追加する。

このとき、gcd2 は $gcd2(i_4) \neq o_4$ となり**失敗**する。

```
FAILED tests/tdd_gcd_tutorial/gcd2/test_4.py::test_t4 - assert 24 == 8
```

そこで、次の実装を作る。

```python
def gcd3(a: int, b: int) -> int:
    # Return the smaller number of a and b.
    c = b
    if a < b:
        c = a
    return c
```

gcd3 はテスト

$$X_1, \ldots, X_4$$

すべて**成功**する。

この実装は少し冗長であり、メソッド `min` を使うほうがわかりやすいので、次のようにリファクタリングする。

```python
# src/tdd_gcd_tutorial/gcd3.py
def gcd3(a: int, b: int) -> int:
    # Return the smaller number of a and b.
    return min(a, b)
```

この実装はテストに成功する。

2.4.5 STEP 5 — 一度では割り切れない場合

次のテストを追加する。

$$X_5 = (i_5, o_5) = ((24, 16), 8)$$

を考える。

このとき、gcd3 は $gcd3(i_5) \neq o_5$ となり**失敗**する。

```
FAILED tests/tdd_gcd_tutorial/gcd3/test_5.py::test_t5 - assert 16 == 8
```

そこでユークリッドの互除法のそこでユークリッドの互除法の 1 ステップだけを行う実装を書く。

```python
# src/tdd_gcd_tutorial/gcd4.py
def gcd4(a: int, b: int) -> int:
    # One step of the Euclidean algorithm.
    # Replace the smaller number with the remainder.
    x = max(a, b)
    y = min(a, b)
    r = x % y
    if r:
        y = r
    return y
```

gcd4 はテスト

$$X_1, \ldots, X_5$$

すべて**成功**する。

この実装は少し冗長であるので、次のようにリファクタリングする。

```python
def gcd4(a: int, b: int) -> int:
    # One step of the Euclidean algorithm.
    # Replace the smaller number with the remainder.
    a, b = max(a, b), min(a, b)
    return a % b if a % b else b
```

この実装はテストに成功する。

2.4.6 STEP 6 — 互いに素

最後に

$$X_6 = (i_6, o_6) = ((24, 7), 1)$$

を追加する。
このとき、gcd4 は $gcd4(i_6) \neq o_6$ となり**失敗**する。

```
FAILED tests/tdd_gcd_tutorial/gcd4/test_6.py::test_t6 - assert 3 == 1
```

そこで ユークリッドの互除法を完全に実装する。

```
# src/tdd_gcd_tutorial/gcd5.py
def gcd5(a: int, b: int) -> int:
    # Straightforward Euclidean algorithm. Time complexity: O(log n).
    while True:
        if b == 0:
            return a
        else:
            r = a % b
            a = b
            b = r
```

gcd5 はテスト

$$X_1, \ldots, X_6$$

すべて**成功**する。
これでユークリッドの互除法のプログラムが完成した。
gcd5 はユークリッドの互除法のプログラムの 1 つであるが、少し複雑である。そこで例えば、以下の gcd6 と gcd7 のような実装は gcd5 と同様にすべてのテストに成功する。

```
# src/tdd_gcd_tutorial/gcd6.py
def gcd6(a: int, b: int) -> int:
    # Overly short Euclidean algorithm. Time complexity: O(log n).
    return a if b == 0 else gcd6(b, a % b)

# src/tdd_gcd_tutorial/gcd7.py
def gcd7(a: int, b: int) -> int:
    # Human-readable Euclidean algorithm. Time complexity: O(log n).
    while b !=0 :
        a, b = b, a % b
    return a
```

さらに次の gcd8 は単純な割り算探索 (brute-force) の例である。

```python
def gcd8(a: int, b: int) -> int:
    # Naive division search (brute-force). Time complexity: O(n).
    m = min(a, b)
    for i in range(m, 0, -1):
        if a % i == 0 and b % i == 0:
            return i
```

これらはリファクタリングの候補である。しかしそれぞれに特徴がある。gcd6 は簡潔しすぎて人間にわかりづらい。gcd8 は計算量が gcd5 と異なる。よって、最も適切なリファクタリングは gcd7 である。

3 プラグマティズムの格率

プラグマティズムの創始者パースは、観念の意味について次のように述べている。

> 我々が持つ概念の対象は何らかの効果を及ぼすと、我々が考えているとして、もしその効果が行動に対しても実際に影響を及ぼしうると想定されるなら、それはいかなる効果であるか、しかと吟味せよ。この吟味によって得られる、こうした効果について我々が持つ概念こそ、当の対象について我々が持つ概念のすべてをなしている。
> CP 5.402

この主張は**プラグマティズムの格率 (Pragmatic Maxim)** と呼ばれる。これは科学的探究において必要な論理学的な原理であるとパースは主張する (どこかにあった。引用箇所を調べる必要がある。もしかしたらそう述べていたのはアブダクションのことだったかもしれない)。この格率は次のように言い換えることができる。

観念の意味はそれが生む観測可能な結果にある

つまり、ある概念を理解するとは、その概念がどのような状況でどのような結果を生むかを知ることである。例えば「硬い」という概念を考える (CP 5.403 CP 5.467)。この概念の意味は

- 叩くと音がする
- 押しても変形しない
- ナイフで傷がつきにくい

といった**観測可能な結果**によって明らかになる。
つまり、概念の意味は、それを区別するテストによって理解される。

4 反証可能性

ポパーは科学理論の条件として**反証可能性 (falsifiability)** を提案した。ポパーによれば、科学理論とは

観測によって誤りである可能性がある理論

である。
例えば次の全称命題を考える。

すべての白鳥は白い

この命題は、黒い白鳥が観測されれば反証される。
つまり、いくつかの全称命題の体系である科学理論とは

ある観測が起これば
誤りであることが示される理論

である。
この意味で、科学理論は常に**テスト可能な予測**を含んでいる。

5　テスト中心主義の定式化へ向けてのいくつかの例

5.1　最大公約数

TDD 開発の例で示した最大公約数のプログラム `gcd1` から `gcd8` について考えてみよう。

5.1.1　振る舞い関数

`gcd` は自然数のペアを入力として受け取り、自然数を出力する関数である。すなわち、**入力集合**を I 、**出力集合**を O とすると、

$$I = \mathbb{N}^2$$
$$O = \mathbb{N}$$

である。`gcd` は入力値 $i \in I$ に対して、出力

$$gcd(i) \in O$$

が存在する。そこで、`gcd` の**振る舞い関数 (behavioral function)** を

$$\mathrm{behavior}_{gcd} : I \to O$$

で定義する。
例えば、`gcd1` の振る舞い関数 $\mathrm{behavior}_{gcd1}$ に対して、テスト X_1 の入力 i_1 を与えると、

$$\mathrm{behavior}_{gcd1}(i_1) = 1$$

となる。これは

入力に対して応答できる

を意味する。

5.1.2 テスト

TDD では通常、各テストに期待値 (expected value) が与えられる。しかし実際には、期待値が事前に分からない場合も存在する。そこでここではテストを次の 2 種類に分類する。

- 観測テスト (Observed test)
- オープンテスト (Open test)

観測テストとは、入力とそれに対応する期待値が与えられているテストである。

$$X_{ob} = (i,\, o) \in I \times O$$

一方、**オープンテスト**とは期待値 (観測値) が与えられていないテストである。ここでは期待値が未定であることを $\perp$ で表す。

$$X_{op} = (i,\, \perp) \in I \times \{\, \perp \,\}$$

したがって、テストは

$$X = (i, o) \in I \times (O \cup \{\perp\})$$

である。

観測テスト $X_{ob} = (i, o)$ が与えられている場合、その期待値を取り除くことで対応するオープンテスト X_{op}

$$(i,\, \perp)$$

を構成できる。

5.1.3 テスト列

一般的にテストには順序がある。そのため、テスト集合ではなく**テスト列**として扱う。テスト列 $\mathbb{X}$ とは、有限個のテストを順序つきで並べたものである。すなわち、

$$\mathbb{X} = (X_1, X_2, \ldots, X_n)$$

である。ここで、$X_j \in I \times (O \cup \perp)$ である。例えば、次のようなテスト列を考える。

$$\mathbb{X} = ((i_1,\, o_1),\, (i_2,\, o_2),\, (i_3,\, o_3),\, (i_4,\, \perp))$$

このとき、1〜3 番目は観測テストであり、4 番目はオープンテストである。

gcd のテスト列 $\mathbb{X} = (X_1,\, X_2,\, X_3,\, X_4,\, X_5,\, X_6)$ を次のように定義する。

テスト	入力 i	出力 o
X_1	(1, 1)	1
X_2	(3, 1)	1
X_3	(24, 8)	8
X_4	(8, 24)	8
X_5	(24, 16)	8
X_6	(24, 7)	1

5.1.4 テスト結果

■5.1.4.1 識別　それぞれの gcd を実行すると、次の結果が出力される。

	i_1: (a,b) = (1, 1)	i_2: (a,b) = (3, 1)	i_3: (a,b) = (24, 8)	i_4: (a,b) = (8, 24)	i_5: (a,b) = (24, 16)	i_6: (a,b) = (24, 7)
gcd1	1	1	1	1	1	1
gcd2	1	1	8	24	16	7
gcd3	1	3	8	8	16	7
gcd4	1	1	8	8	8	3
gcd5	1	1	8	1	8	1
gcd6	1	1	8	1	8	1
gcd7	1	1	8	1	8	1
gcd8	1	1	8	1	8	1

ここから次のことがわかる。

1. gcd5 と gcd6 はテスト列 $\mathbb{X}$ に対して、同じ結果を出力するので、その振る舞いから区別することができない。このとき「gcd5 はテスト列 $\mathbb{X}$ に対して gcd6 と**識別同値 (discriminative equivalent)** である」と呼ぼう。これは**振る舞い同値 (behavioral equivalent)** と呼んでもいい。同様に gcd5 はテスト列 $\mathbb{X}$ に対して gcd7 および gcd8 とも識別同値である。
2. gcd1 と gcd2 はテスト X_3 においてそので出力結果が異なっているので、振る舞いから区別することができる。つまり、gcd1 はテスト列 $\mathbb{X}$ に対して gcd2 と識別同値ではない。同様に gcd1 は gcd3、gcd4、gcd5 とも識別同値ではない。

■3.1.4.2 検証　テストを実行すると次の結果になる。

	X_1: (i_1, o_1) $= ((1, 1), 1)$	X_2: (i_2, o_2) $= ((3, 1), 1)$	X_3: (i_3, o_3) $= ((24, 8), 8)$	X_4: (i_4, o_4) $= ((8, 24), 8)$	X_5: (i_5, o_5) $= ((24, 16), 8)$	X_6: (i_6, o_6) $= ((24, 7), 1)$
gcd1	True	True	False	False	False	True
gcd2	True	True	True	False	False	False
gcd3	True	True	True	True	False	False
gcd4	True	True	True	True	True	False
gcd5	True	True	True	True	True	True
gcd6	True	True	True	True	True	True
gcd7	True	True	True	True	True	True
gcd8	True	True	True	True	True	True

ここから次のことがわかる。

1. `gcd5` はテスト列 $\mathbb{X}$ に対して、すべてのテストで出力が期待値と一致している。このとき「`gcd5` はテスト列 $\mathbb{X}$ によって**検証されている (verified)** 」と呼ぼう。同様に `gcd6` かつ `gcd7` かつ `gcd8` もテスト列 $\mathbb{X}$ によって検証されている。
2. `gcd5` と `gcd6` はいずれもテスト列 $\mathbb{X}$ で検証されている。このとき「`gcd5` はテスト列 $\mathbb{X}$ に対して `gcd6` と**検証同値 (verification equivalent)** である」と呼ぼう。これは**仕様同値 (specification equivalent)** と呼んでもいい。同様に `gcd5` はテスト列 $\mathbb{X}$ に対して `gcd7` および `gcd8` とも検証同値である。
3. `gcd1` はテスト列 $\mathbb{X}$ に対して、少なくとも 1 つのテストで期待値と一致しない。例えば、`gcd1` はテスト X_3 において期待値と不一致 (`False`) である。このとき「`gcd1` はテスト列 $\mathbb{X}$ によって**反証されている (falsified)** 」と呼ぼう。同様に `gcd2` かつ `gcd3` かつ `gcd4` はテスト列 $\mathbb{X}$ によって反証されている。
4. さらに前述の識別と比較すると、`gcd` がテスト列 $\mathbb{X}$ に対して `gcd'` と検証同値であるならば、`gcd` は $\mathbb{X}$ に対して `gcd'` と識別同値であることがわかる。

5.2 プラグマティズムの例

「硬い」と「柔らかい」という概念を考える。これらの概念を理解するためには、次のようなテストを考えることができる。

- ナイフで引っかく

プラグマティズムの格率とは、次を満たす原理であるとみなせる。すなわち、概念の対象はテストによって**識別可能**であり、概念の意味とは、その概念がテストに対して示す振る舞いである。

意味 = テストが示す振る舞い

これらのテストの結果によって、「硬い」と「柔らかい」は概念として区別される。

つまり、

概念はそれを区別するテストによって理解される。

プラグマティズムの格率が直接述べているのは、概念が識別可能であることである。すなわち、その概念を用いることで、どのような状況でどのような違いが生じるかを区別できなければならないという点である。

ただし、この段階では実際に何が起こるかまではまだ確定していない。概念を用いることで「このような結果になるだろう」という予測が得られるが、その予測が正しいかどうかはまだ分からない。

その予測が正しいかどうかは、実験や観測によって確かめられる。ここで初めて、概念に基づく予測が検証される。

科学的探究において実験が不可欠であるのはこのためである。概念を実験によって検証するためには、まずその概念が識別可能であることが必要である。プラグマティズムの格率は、この識別可能性を概念の意味の条件として示しているのである。

5.3 科学理論の例

科学理論もまた、テストによって理解される対象である。例えば、次の命題 P を考える。

すべての白鳥は白い

この命題は、白鳥を観測することによって調べることができる。例えば、次のようなテストを考える。

白鳥を観測し、その色を確認する

これまでおこなわれた観測の結果として、観測テスト列

$$\mathbb{X}_n = (X_1, \ldots, X_n)$$

が得られているとする。ここで、各テスト X_i は白鳥を観測し、その色を確認するものである。

これまで観測された白鳥がすべて白であれば、命題 P は観測結果と矛盾しない。すなわち、命題 P はテスト列 $\mathbb{X}_n$ によって検証されている。

次に、観測がさらに続けられる場合を考える。最初の $N-1$ 回は観測テストであり、N 回目がオープンテストであるテスト列 $\mathbb{X}_N$

$$\mathbb{X}_N = (X_1, \ldots, X_n, X_{n+1}, \ldots, X_{N-1}, X_N)$$

を考えよう。ここで、$X_i \in I \times O\,(i = 1, \ldots, N-1)$ であり、$X_N \in I \times \{\bot\}$ である。

命題 P はオープンテスト X_N の入力 i_N に対して次のような予測を与える。

$$\mathrm{behavior}_P(i_N) = \text{the swan}\, i_N \text{ is white}$$

もしこの観測において、白くない白鳥 (例えば黒い白鳥) が観測された場合、

$$\mathrm{behavior}_P(i_N) \neq o_N$$

となり、命題 P はテスト列 $\mathbb{X}_N$ によって反証される。

このように科学理論は、観測によって反証される可能性を持つ。言い換えれば、科学理論はそれを反証する可能性のあるテストによって特徴づけられる。

理論はそれを反証する可能性のあるテストによって特徴づけられる

この意味で、科学理論は常にまだ観測されていない現象についての予測を含んでいる。

6　テスト中心主義

6.1　基本アイデア

対象を次の 3 種類に分類する。

- Concept
- Proposition
- Theory

テスト中心主義の基本的な立場は次の通りである。

対象はテストによって理解される

ここでいう「テスト」とは、対象に対して何らかの操作や観測を行い、その結果を得る手続きのことである。
概念の場合には、テストはその概念がどのような観測可能な結果をもたらすかを調べる操作である。
命題の場合には、テストはその命題が成立するかどうかを確認する観測である。
理論 (命題の体系) の場合には、テストはその理論が与える予測を観測と比較する手続きである。
このように、概念・命題・理論はいずれも、テストによってその意味や内容が明らかになる。

6.2　テストの役割

テストには 2 つの役割がある。それは**構成のためのテスト**と**理解のためのテスト**である。

6.2.1　構成のためのテスト

構成のためのテストとは、TDD でおこなわれるテストである。つまり、「仕様 →テスト → 実装」である。

6.2.2　理解のためのテスト

理解のためのテストとは、ある対象 (概念・命題・理論) を理解するためにおこなわれるテストである。つまり、「対象→ テスト → 振る舞い」である。「理論を理解する ＝ それを区別するテストを作れる」ことである。

7　定式化

テスト中心主義を形式化する。

7.1 公理 1(対象)

対象の集合を

$$\mathcal{T}$$

とする。
対象 $T \in \mathcal{T}$ は次のいずれかの種類に属する。

- 概念 (Concept)
- 命題 (Proposition)
- 理論 (Theory)

7.2 公理 2(振る舞い関数)

入力集合 I と出力集合 O が存在する。
各対象

$$T \in \mathcal{T}$$

は振る舞い関数

$$\mathrm{behavior}_T : I \to O$$

を持つ。

7.3 公理 3(テスト)

対象 T に対するテスト X は

$$(i,\, o) \in I \times (O \cup \{\bot\})$$

である。ここで、

- $(i,\, o) \in I \times O$ を観測テスト (Observed test)
- $(i,\, \bot) \in I \times \{\bot\}$ をオープンテスト (Open test)

という。オープンテストは実際の観測が未確定であるテストを表す。

7.4 公理 4(テスト列)

有限個のテストを順序つきで並べたテスト列 $\mathbb{X}$ という。

$$\mathbb{X} = (X_1,\, \ldots, X_n)$$

ここで、各 X_i はテストであり、

$$X_i \in I \times (O \cup \{\bot\})$$

を満たす。
テスト列 $\mathbb{X}$ の入力集合を

$$inputs(\mathbb{X}) = \{\, p_i : X_i \to I \text{ is the projection} \mid X_i \in \mathbb{X} \,\}$$

と定義する。

7.5 公理 5(意味)

対象 T のテスト列 $\mathbb{X}$ に関する**意味** $\mathrm{Meaning}_{\mathbb{X}}(T)$ とは

$$\mathrm{Meaning}_{\mathbb{X}}(T) = (\mathrm{behavior}_T(i_1), \ldots, \mathrm{behavior}_T(i_n))$$

ここで $i_j \in inputs(\mathbb{X})$ である。

7.6 公理 6(検証可能性)

テスト列 $\mathbb{X}$ **検証可能**であるとは、すべてのテスト $X_i \in \mathbb{X}$ が観測テストであることをいう。

7.7 公理 7(識別同値性)

対象 $T, S \in \mathcal{T}$ がテスト列 $\mathbb{X}$ について**識別同値**であるとは

$$\forall i \in inputs(\mathbb{X}) : \mathrm{behavior}_T(i) = \mathrm{behavior}_S(i)$$

である。このとき、

$$T \sim_{\mathbb{X}} S$$

と書く。
これは

与えられたテスト列では区別できない

ことを意味する。

7.8 公理 8(検証同値性)

対象 $T \in \mathcal{T}$ が検証可能なテスト列 $\mathbb{X}$ で**検証される**とは、

$$T \vDash \mathbb{X}$$

と書き，テスト列 $\mathbb{X}$ のすべてのテスト $X = (i, o)$ に対して、

$$\mathrm{behavior}_T(i) = o$$

である。
対象 $T, S \in \mathcal{T}$ がテスト列 $\mathbb{X}$ について**検証同値**であるとは

$$T \vDash \mathbb{X} \quad \wedge \quad S \vDash \mathbb{X}$$

である。このとき、

$$T \approx_{\mathbb{X}} S$$

と書く。
つまり

両方とも与えられたテスト列をパスする

ことである。
対象 $T, S \in \mathcal{T}$ が検証同値ならば、識別同値である。つまり、

$$T \approx_{\mathbb{X}} S \Rightarrow T \sim_{\mathbb{X}} S$$

である。

7.9 公理 9(理解)

主体が対象 $T \in \mathcal{T}$ をテスト列 $\mathbb{X}$ に関して**理解する**とは任意の入力 $i \in inputs(\mathbb{X})$ に対して、

$$\mathrm{behavior}_T(i)$$

を予測 (または決定) できる能力を持つことである。

理解 = テスト応答能力

7.10 公理 10(複雑度)

対象の集合 $\mathcal{T}$ 上に複雑度関数

$$C : \mathcal{T} \to \mathbb{R}_{\geq 0}$$

が定義される。

7.11 公理 11(リファクタリング)

対象 $T, S \in \mathcal{T}$ に対して

$$T \approx_{\mathbb{X}} S \wedge C(S) \leq C(T)$$

ならば、S は T の**リファクタリング**である。

8 例

8.1 最大公約数: 検証可能な対象の例

`gcd` を命題とみなす。

- 対象の集合:

$$\mathcal{T}_{\mathrm{gcd}} = \{\, \mathrm{gcd1},\, \mathrm{gcd2},\, \mathrm{gcd3},\, \mathrm{gcd4},\, \mathrm{gcd5},\, \mathrm{gcd6},\, \mathrm{gcd7},\, \mathrm{gcd8} \,\}$$

- 検証可能なテスト列:

$$\mathbb{X} = (X_1,\, X_2,\, X_3,\, X_4,\, X_5,\, X_6)$$

各テストは

$$X_k = (i_k,\, o_k)$$

という形をとり

- 入力 $i_k = (i_{k_1},\, i_{k_2}) \in \mathbb{N}^2$
- 観測 $o_k \in \mathbb{N}$

を表す。

- 検証可能性:

すべての対象

$$T \in \mathcal{T}_{\mathrm{gcd}}$$

はテスト列 $\mathbb{X}$ に対して検証可能である。

- 識別同値性:

$$
\begin{aligned}
&\text{gcd1} \nsim_{\mathbb{X}} \text{gcd2}\\
&\text{gcd1} \nsim_{\mathbb{X}} \text{gcd3}\\
&\text{gcd1} \nsim_{\mathbb{X}} \text{gcd4}\\
&\text{gcd1} \nsim_{\mathbb{X}} \text{gcd5}\\
&\text{gcd2} \nsim_{\mathbb{X}} \text{gcd3}\\
&\text{gcd2} \nsim_{\mathbb{X}} \text{gcd4}\\
&\text{gcd2} \nsim_{\mathbb{X}} \text{gcd5}\\
&\text{gcd3} \nsim_{\mathbb{X}} \text{gcd4}\\
&\text{gcd3} \nsim_{\mathbb{X}} \text{gcd5}\\
&\text{gcd5} \sim_{\mathbb{X}} \text{gcd6}\\
&\text{gcd5} \sim_{\mathbb{X}} \text{gcd7}\\
&\text{gcd5} \sim_{\mathbb{X}} \text{gcd8}
\end{aligned}
$$

- 検証同値性:

$$
\begin{aligned}
&\text{gcd1} \not\approx_{\mathbb{X}} \text{gcd2}\\
&\text{gcd1} \not\approx_{\mathbb{X}} \text{gcd3}\\
&\text{gcd1} \not\approx_{\mathbb{X}} \text{gcd4}\\
&\text{gcd1} \not\approx_{\mathbb{X}} \text{gcd5}\\
&\text{gcd2} \not\approx_{\mathbb{X}} \text{gcd3}\\
&\text{gcd2} \not\approx_{\mathbb{X}} \text{gcd4}\\
&\text{gcd2} \not\approx_{\mathbb{X}} \text{gcd5}\\
&\text{gcd3} \not\approx_{\mathbb{X}} \text{gcd4}\\
&\text{gcd3} \not\approx_{\mathbb{X}} \text{gcd5}\\
&\text{gcd5} \approx_{\mathbb{X}} \text{gcd6}\\
&\text{gcd5} \approx_{\mathbb{X}} \text{gcd7}\\
&\text{gcd5} \approx_{\mathbb{X}} \text{gcd8}
\end{aligned}
$$

- 意味:

$$
\begin{aligned}
\text{Meaning}_{\mathbb{X}}[\text{gcd5}] &= (\text{behavior}_{\text{gcd5}}(i_1), \ldots, \text{behavior}_{\text{gcd5}}(i_6))\\
&= (1,\ 1,\ 8,\ 1,\ 8,\ 1)\\
&= \text{Meaning}_{\mathbb{X}}[\text{gcd6}] = \text{Meaning}_{\mathbb{X}}[\text{gcd7}] = \text{Meaning}_{\mathbb{X}}[\text{gcd8}]
\end{aligned}
$$

つまり、`gcd5`、`gcd6`、`gcd7`、`gcd8` はテスト列に関して同じ意味をもつ。

- 理解:

主体が `gcd5` を理解するとは、任意の入力 $i \in \mathit{inputs}(\mathbb{X})$ に対して、

$$\mathrm{behavior}_{\mathrm{gcd5}}(i)$$

を予測できる能力を持つことである。

- リファクタリング:

`gcd5` から `gcd7` への操作はリファクタリングである。

$$\mathrm{gcd5} \to \mathrm{gcd7}$$

8.2 プラグマティズムの格率: 硬い

- 対象の集合:

$$\mathcal{T}_{\mathrm{hs}} = \{\ \mathrm{hard}\ ,\ \mathrm{soft}\ \}$$

- テスト列:

$$\mathbb{X} = ((\ \text{knife-edge test}, \perp))$$

このテスト列はオープンテストのみである。

- 識別可能性 (プラグマティズムの格率):

プラグマティズムの格率によれば、概念の意味はその概念がもたらす**想定される効果**によって与えられる。「硬い」とはナイフの刃で引っかいたときの振る舞いで定義される。

$$\begin{aligned}\mathrm{behavior}_{\mathrm{hard}}(\text{knife-edge test}) &= \mathrm{resist}\\ \mathrm{behavior}_{\mathrm{soft}}(\text{knife-edge test}) &= \mathrm{scratch}\end{aligned}$$

- 識別同値性:

$$\mathrm{behavior}_{\mathrm{hard}}(\text{knife-edge test}) = \mathrm{resist} \neq \mathrm{scratch} = \mathrm{behavior}_{\mathrm{soft}}(\text{knife-edge test})$$

したがって

$$\mathrm{hard} \approx_{\mathbb{X}} \mathrm{soft}$$

となり、両者はこのオープンテストによって識別可能である。

- 意味:

$$\begin{aligned}\mathrm{Meaning}_{\mathbb{X}}[\mathrm{hard}] &= (\mathrm{behavior}_{\mathrm{hard}}(\text{knife-edge test})) = (\mathrm{resist})\\ \mathrm{Meaning}_{\mathbb{X}}[\mathrm{soft}] &= (\mathrm{behavior}_{\mathrm{soft}}(\text{knife-edge test})) = (\mathrm{scratch})\end{aligned}$$

ここでは観測は導入されていない。したがってこれは**検証ではなく識別の例**である。

この例はプラグマティズムの格率「概念の意味はその概念の実践的効果によって決まる」をテスト中心主義の枠組みで表現したものである。

8.3 科学理論: 反証可能性

次に、科学理論を考える。

- 対象の集合:

$$\mathcal{T}_{\text{swan}} = \{\, T \,\}$$

ここで、$T =$「すべての白鳥は白い」という全称命題である。

- テスト列:

$$\mathbb{X} = ((i_1,\, \text{white}),\, (i_2,\, \text{white}), \ldots, (i_{n-1},\, \text{white}),\, (i_n,\, \bot))$$

を `n-1` 番目まで観測テストであり、`n` 番目がオープンテストであるとする。ここでのテスト方法は

白鳥を観測し、その色を確認する

というものとする。
命題 T はテスト列のすべての入力 $i \in \mathit{inputs}(\mathbb{X})$ に対して、

$$\text{behavior}_T(i) = \text{white}$$

を予測する。

- 検証可能性

`n` 番目のオープンテストで観測 o_n が出たとする。もし、

$$o_n = \text{white}$$

が成立すれば、テスト列 $\mathbb{X}$ のすべてのテストで $\text{behavior}_T(i) = o_i$ であるので、

$$T \vDash \mathbb{X}$$

となる。
このとき命題 T はテスト $\mathbb{X}$ によって検証された。

- 反証可能性

もし、オープンテストの観測結果が

$$o_n = \text{black}$$

であるならば、

$$\mathrm{behavior}_T(n_i) \neq o_n$$

となる。
したがって

$$T \not\models \mathbb{X}$$

である。このとき命題 T はテスト列 $\mathbb{X}$ よって反証される。
したがって、科学理論は

```
観測と一致しない結果が現れる可能性を持つ
```

という意味で反証可能である。
この構造はテスト中心主義では次のように表される。

```
反証 = behavior と observation の不一致
```

9 圏論での定式化

9.1 公理 1. テスト圏

テスト圏

$$\mathcal{X}$$

を次のように定義する。

- 対象

```
finite test sequences
```

$$\mathbb{X} = ((i_1, o_1), \ldots, (i_n, o_n))$$

ここで、$i_k \in I$ であり、$o_k \in O \cup \{\bot\}$ である。

- 射

$$\mathbb{X} \to \mathbb{Y}$$

は

```
prefix extension
```

つまり

$$\mathbb{X} \preceq \mathbb{Y}$$

である ($\mathbb{X}$ が $\mathbb{Y}$ の前半部分列)。
これは

```
テストが順序付きで増える
```

ことを意味する。

9.2 公理 2. 振る舞い関手

対象 $T \in \mathcal{T}$ に対して振る舞い関手

$$F_T : \mathcal{X}^{op} \to \mathbf{Set}$$

を定義する。

$$F_T(\mathbb{X}) = \{f : inputs(\mathbb{X}) \to O \mid f(i) = \mathrm{behavior}_T(i)\}$$

これはテスト列上での振る舞いのグラフである。

- 射に対する作用

$$\mathbb{X} \preceq \mathbb{Y}$$

に対して

```
restriction(制限写像)
```

これは

```
テストが増えるほど観測される振る舞いが増える
```

ことを意味する。

9.3 公理 3. 意味

対象 $T \in \mathcal{T}$ の圏論的意味は

$$\mathrm{Meaning}(T) = F_T$$

である。

つまり

```
meaning = behavior presheaf
```

9.4 公理 4. 識別同値

対象 $T, S \in \mathcal{T}$ が識別同値であるとは

$$F_T = F_S$$

であることをいう。
つまり

$$\forall i \in I, \quad \mathrm{behavior}_T(i) = \mathrm{behavior}_S(i)$$

これは

すべてのテストに対して区別できない

ことを意味する。

9.5 公理 5. 検証

対象 $T \in \mathcal{T}$ がテスト列 $\mathbb{X}$ を満たすとは

$$T \vDash \mathbb{X}$$

と書き、

$$o_k \neq \bot \Rightarrow \mathrm{behavior}_T(i_k) = o_k$$

を満たすこととする。
これは

観測があるテストのみ一致を要求する

ことを意味する。

9.6 公理 6. Test-Centrism 圏

Test-Centrism 圏を

$$\mathbf{TC} = \mathbf{Set}^{\mathcal{X}^{op}}$$

と定義する。

これは

```
presheaf topos
```

である。

9.7 Test-Centrism での Yoneda の定理

Yoneda の補題より

$$\mathrm{Nat}(y(\mathbb{X}), F_T) \cong F_T(\mathbb{X})$$

が成り立つ。
ここで、$y(\mathbb{X}) = \mathrm{Hom}(-, \mathbb{X})$ であり、Nat は自然変換の集合である。

9.7.1 定理

対象 T はその振る舞い関手 F_T によって完全に決定される。すなわち

$$F_T = F_S \iff T \sim S \iff \forall i \in I, \quad \mathrm{behavior}_T(i) = \mathrm{behavior}_S(i)$$

これは

対象 = テスト応答の総体

であることを意味する。

9.8 理解の圏論的解釈

対象 $T \in \mathcal{T}$ をテスト列 $\mathbb{X}$ に関して理解するとは、

$$F_T(\mathbb{X})$$

を与えることである。これは

テスト列に対する応答を決定できる

ことを意味する。

10 トポスの内部論理

Test-Centrism 圏

$$\mathbf{TC} = \mathbf{Set}^{\mathcal{X}^{op}}$$

は presheaf トポスである。したがって内部論理は**直観主義論理**である。

10.1 真理値対象

真理値対象

$$\Omega$$

は subobject classifier である。

presheaf トポスでは

$$\Omega(\mathbb{X})$$

は

```
上の sieve
```

である。

10.2 sieve の意味

ここで sieve とは

```
𝕏 に入る射の集合
```

であり、ここでは

```
𝕏 の prefix(部分テスト列) の集合
```

に対応する。さらに sieve は

```
prefix に関して閉じている
(sieve = future-closed)
```

すなわち

```
あるテスト列で成り立つなら
それ以前のテストでも成り立つ
```

10.3 命題の意味

命題 P の真理値は

$$\Omega(\mathbb{X})$$

の元、すなわち sieve である。

これは

P が成立するだけでなく
その後のすべての拡張でも成立する集合

を意味する。
ただし重要なのは

テストの拡張に対して安定

であることである。

10.4 真理の意味

命題 P がテスト列 $\mathbb{X}$ において真であるとは

$\mathbb{X}$ を含む sieve に属する

ことである。
これは

P は $\mathbb{X}$ で成立し
かつその後のテスト拡張でも崩れない

ことを意味する。

10.5 論理結合子

10.5.1 AND(論理積)

$$P \wedge Q$$

両方の条件を満たすテスト列の共通部分

10.5.2 OR(論理和)

$$P \vee Q$$

P ∨ Q は、現在のテスト列において
P または Q が成立すること

ここでは単なる「どちらかが成り立つ」ではなく、拡張で保証される必要がある。

10.5.3　含意

$$P \Rightarrow Q$$

任意のテスト拡張において
`P` が成り立つなら `Q` も成り立つ

これは非常に重要で

安定な推論

を意味する。

10.5.4　否定

$$\neg P$$

とは

$$P \Rightarrow \bot$$

であり

どのテスト拡張においても
`P` が成立しない

ことを意味する。

10.6　排中律が成立しない理由

一般に

$$P \vee \neg P$$

は成立しない。
理由:

まだ十分なテストが行われていない状態

が存在するためである。
つまり

P も ¬P も確定していない

これは

科学的探究の途中状態

に対応する。

10.7 哲学的解釈

テスト中心主義の内部論理は

テスト依存的直観主義論理

である。
ここでは

- 真理 = すべての将来のテストにおいても崩れない成立
- 命題 = テスト列に依存する性質

となる。

10.8 科学哲学との対応

この構造は

- Karl Popper の反証主義
- Charles Sanders Peirce のプラグマティズム

と一致する。
特に

否定 = 反証可能性

という解釈が成立する。

- テスト列 = 知識の蓄積
- 拡張 = 実験の追加
- 安定性 = 反証されない

11 Kripke 意味論

Test-centrism の内部論理は、**Kripke 意味論**としても解釈できる。

11.1 Kripke フレーム

Kripke フレームを

$$(\mathcal{X}, \preceq)$$

とする。

- 世界：テスト列 $\mathbb{X}$
- 到達関係：prefix 関係 $\preceq$

テストが拡張されるほど未来へ進む

11.2 強制関係 (forcing)

命題 P に対して

$$\mathbb{X} \Vdash P$$

を「$\mathbb{X}$ において P が成立する」と定義する。
このとき重要な条件は

単調性 (`monotonicity`)

である。

$$\mathbb{X} \preceq \mathbb{Y}, \quad \mathbb{X} \Vdash P \Rightarrow \mathbb{Y} \Vdash P$$

つまり

一度成立した命題は
テストが増えても崩れない

11.3 論理結合子

Kripke 意味論では論理は次のように解釈される。

11.3.1 AND(論理積)

$$\mathbb{X} \Vdash P \wedge Q \iff \mathbb{X} \Vdash P \text{ and } \mathbb{X} \Vdash Q$$

11.3.2　OR(論理和)

$$\mathbb{X} \Vdash P \vee Q$$

とは

```
P ∨ Q は、現在のテスト列において
P または Q が成立する
```

ただし成立性は将来に対して安定である必要がある。

11.3.3　含意

$$\mathbb{X} \Vdash P \Rightarrow Q$$

とは

$$\forall \mathbb{Y} \succeq \mathbb{X}, \quad \mathbb{Y} \Vdash P \Rightarrow \mathbb{Y} \Vdash Q$$

11.3.4　否定

$$\mathbb{X} \Vdash \neg P$$

とは

$$\forall \mathbb{Y} \succeq \mathbb{X}, \quad \mathbb{Y} \nVdash P$$

11.4　トポスとの一致

この Kripke 意味論は、前節のトポス内部論理と一致する。
対応は次の通り

トポス	Kripke
sieve	上に閉じた真理集合
安定性	単調性
真理	すべての拡張で成立

したがって

```
Test-centrism は
presheaf トポスとして定式化され
その内部論理は Kripke 意味論として解釈できる
```

である。

11.5 哲学的意味

この構造の意味は明確である。

```
世界 = テストの進行状態
知識 = テストの蓄積
真理 = 将来のテストでも崩れない命題
```

これは

- 概念 (実践的帰結)
- プログラム (テストの安定性)
- 科学理論 (反証可能性)

を統一的に説明する。

12 社会理論

テスト中心主義の社会理論はまだ全然考えられていない。以下はアイディアである。

- 従来の社会変革は「理論→ 実践 (革命・政策・運動)」であった。
- しかし、このやり方はあまりに問題が多い。
  - 失敗のコストが大きすぎる
  - 変更単位が大きすぎる
  - 検証がおざなりになる
  - 一度実行した社会変革は元に戻せない
- そこで TC では「理論→テスト→実践」とする。つまり理論を作ったならば、テストを書かなければならない。そしてテストが失敗したときに初めて実践をおこなうようにする。
- 実践はテストを通す最小の変更のみを許容する。
- リファクタリングもおこなう。このとき複雑さをより小さくするということに特に留意しなければならない。

13 結論

このでは、対象をテストによって理解する立場として**テスト中心主義 (Test-centrism)** を提案した。
テスト中心主義では、対象は入力に対する振る舞いとして与えられ、テストは入力と (存在する場合の) 観測

値からなる順序つきの有限列として定義される。このとき、テストに関して次の二つの基本関係が導入される。

- **識別可能性**: テストの入力に対して対象の振る舞いが定義されること
- **検証可能性**: 対象の振る舞いと観測結果を比較できること

この区別に基づき、基本概念は次のように定式化される。

- **意味**: テスト列に制限された振る舞い
- **識別同値性**: すべてのテスト入力に対する振る舞いの一致
- **検証同値性**: すべての観測テストをともに満たすこと
- **理解**: テスト列に対する応答を予測できる能力

またソフトウェア開発におけるリファクタリングは、この枠組みでは次のように理解できる。

- **リファクタリング**: 複雑性を減少させる検証同値性を保つ変換

このようにテスト中心主義は、

- ソフトウェア (プログラムの振る舞い)
- 科学理論 (観測による検証)
- 哲学 (概念の意味)

を共通の構造のもとで扱うことを可能にする。

したがって、テストという概念は単なるソフトウェア開発の技法ではなく、対象を理解するための一般的な方法論として捉えることができる。

テスト中心主義 (Test-centrism) を圏論的に形式化した。圏論的には、テスト列の圏に対する presheaf として対象の意味が与えられ、Test-Centrism 圏はトポス構造を持つ。この内部論理は直観主義論理であり、Kripke 意味論と一致する。そこでは

真理 = テスト拡張に対して安定な成立

として理解される。

この視点から、従来異なる領域とされてきた次の対象が統一される:

- 概念: 状況に対する実践的効果
- ソフトウェア: テストに対する振る舞い
- 科学理論: 実験に対する予測

さらにこの理論は、

- Peirce のプラグマティズム
- Popper の反証主義

を共通の形式のもとに統合する。

最終的に、テスト中心主義は次の原理に要約される:

```
意味 = テストへの応答
真理 = テストに対して安定な成立
理解 = テストに応答できる能力
```

したがってテストとは、単なるソフトウェア開発の技法ではなく、対象を理解し、意味を与え、理論を評価するための**一般的な認識の枠組み**である。